INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DE RONDÔNIA

Curso Superior de Tecnologia em Sistemas para Internet

Mente Saudável

Sistema de gestão para clínica de psicologia

Documentação do projeto integrador

Disciplinas	Desenvolvimento Web III + Banco de Dados II
Turma	2025.1
Tecnologias	Python, Flask, PostgreSQL, HTML, CSS e JavaScript
Ano	2026

Equipe

Integrante 1: _____

Integrante 2: _____

Integrante 3: _____

Integrante 4: _____

Substitua os nomes acima pelos integrantes da equipe antes da entrega.

1. Introdução

O projeto **Mente Saudável** consiste em uma Web API REST desenvolvida em Python com o microframework Flask, integrada a um banco de dados PostgreSQL. O sistema apoia a rotina de uma clínica de psicologia, permitindo gerenciar pacientes, profissionais, consultas e evoluções clínicas, além de gerar relatórios de atendimento.

A solução aplica conceitos de normalização, integridade referencial, check constraints, stored procedures, views com JOIN, triggers e troca de mensagens no formato JSON, conforme os requisitos das disciplinas Desenvolvimento Web III e Banco de Dados II.

2. Objetivo

Desenvolver uma aplicação web com API REST capaz de realizar operações CRUD sobre os cadastros da clínica, persistindo os dados em um banco relacional bem estruturado, com validações no front-end, no back-end e no próprio banco, além de disponibilizar relatórios construídos a partir de views e stored procedures.

3. Descrição do sistema

O sistema é composto por quatro módulos principais, todos acessíveis pela API e pelo painel web:

- **Pacientes:** CRUD completo, busca por nome, filtro por status e resumo individual com total de consultas, consultas realizadas, consultas agendadas, humor médio e última consulta.
- **Psicólogos:** cadastro com CRP, especialidade, valor padrão da sessão e situação (ativo/inativo).
- **Consultas:** agendamento com verificação de conflito de horário, modalidade (presencial ou online), duração, valor e status.
- **Evoluções:** prontuário de cada consulta, com nota de humor (1 a 10), relato clínico e encaminhamento.

O painel web exibe indicadores gerais, próximas consultas, distribuição de atendimentos por especialidade e tabelas com ações de cadastro, edição e exclusão.

4. Arquitetura da solução

A figura a seguir sintetiza a arquitetura: o cliente (painel web ou Postman) envia requisições HTTP com JSON para a API Flask, que valida os dados, executa a lógica de negócio e acessa o PostgreSQL por meio do driver psycopg2.

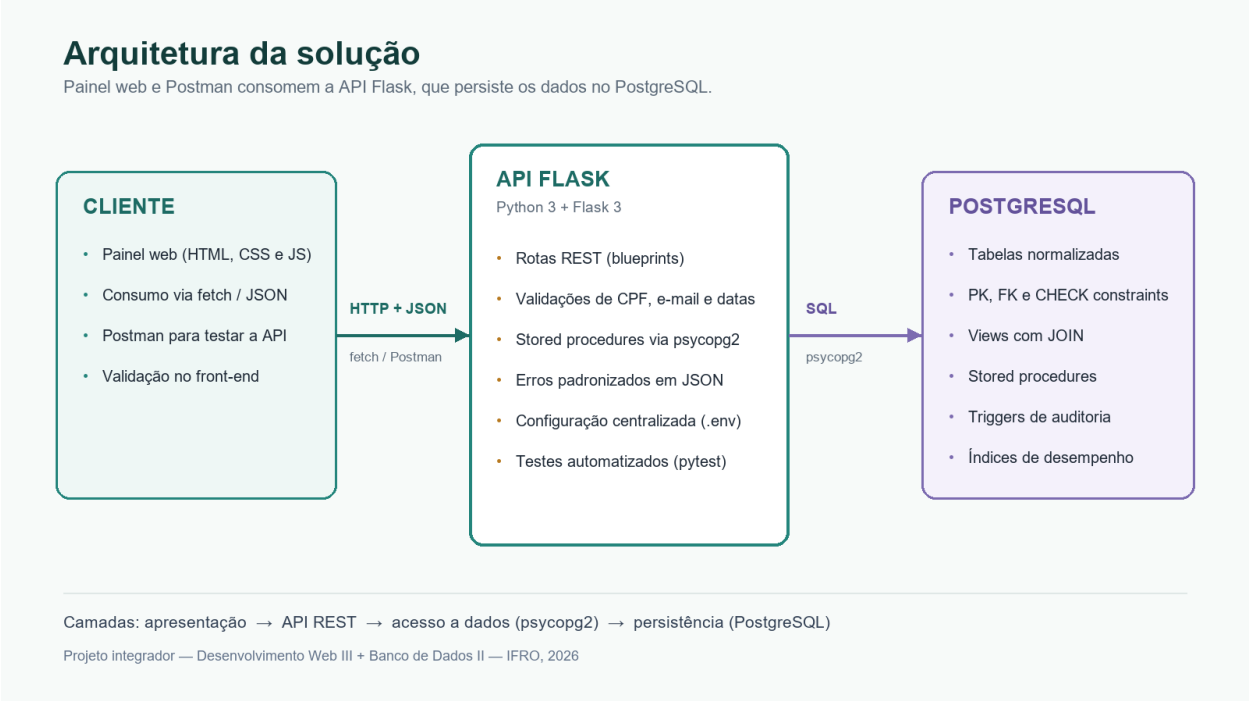


Figura 1 — Arquitetura da solução em camadas.

A API está organizada em blueprints (pacientes, psicólogos, consultas, evoluções, relatórios e health check), com validações reutilizáveis, tratamento centralizado de erros e configuração centralizada em um arquivo `.env`. O acesso ao banco fica isolado no módulo `app/db.py`.

5. Modelo de dados

O banco foi modelado seguindo a normalização (3ª forma normal), evitando redundância. A tabela de evoluções referencia apenas a consulta; os dados do paciente e do psicólogo são obtidos por JOIN.

5.1 Tabelas, chaves e restrições

Tabela	Chave primária	Chaves estrangeiras	Principais regras
psicologos	id_psicologo	—	CRP único (formato UF/número); e-mail válido; valor da sessão > 0
pacientes	id_paciente	—	CPF único com 11 dígitos; e-mail válido; nascimento não futuro; status ativo/inativo
consultas	id_consulta	id_paciente → pacientes; id_psicologo → psicologos	Duração entre 15 e 180 min; modalidade e status restritos; valor ≥ 0
evolucoes	id_evolucao	id_consulta → consultas (única)	Humor entre 1 e 10; relato com no mínimo 10 caracteres
auditoria_pacientes	id_auditoria	—	Registra INSERT/UPDATE/DELETE com snapshots em JSONB

5.2 Check constraints

Foram implementadas mais de 20 check constraints, entre elas: `ck_pacientes_cpf_formato` (CPF com 11 dígitos), `ck_pacientes_nascimento` (data entre 1900 e a data atual), `ck_pacientes_status`, `ck_psicologos_crp_formato`, `ck_psicologos_valor_positivo`, `ck_consultas_duracao` (15 a 180 minutos), `ck_consultas_modalidade`, `ck_evolucoes_humor` (1 a 10) e `ck_evolucoes_relato_minimo`. Um índice único parcial impede dois agendamentos ativos no mesmo horário para o mesmo psicólogo, liberando o horário quando a consulta é cancelada.

5.3 Comentários das colunas no script do banco

Conforme exigido, o script `database/01_schema.sql` descreve cada coluna em comentário ao lado da definição:

```
CREATE TABLE pacientes (
    id_paciente     SERIAL        PRIMARY KEY,
    nome            VARCHAR(120)  NOT NULL,                          -- Nome completo do paciente
    cpf             CHAR(11)      NOT NULL UNIQUE,                   -- CPF sem pontuacao (11 digitos)
    email           VARCHAR(160)  NOT NULL UNIQUE,                   -- E-mail de contato do paciente
    telefone        VARCHAR(20),                                     -- Telefone com DDD (opcional)
    data_nascimento DATE          NOT NULL,                          -- Data de nascimento do paciente
    genero          VARCHAR(20)   NOT NULL DEFAULT 'nao_informado',  -- Genero declarado pelo paciente
    cidade          VARCHAR(80),                                     -- Cidade onde o paciente reside
    observacoes     TEXT,                                            -- Observacoes gerais do cadastro
    status          VARCHAR(15)   NOT NULL DEFAULT 'ativo',          -- Situacao do cadastro
    criado_em       TIMESTAMPTZ   NOT NULL DEFAULT now(),            -- Data/hora de cadastro
    atualizado_em   TIMESTAMPTZ,                                     -- Ultima alteracao
    ...
);
```

6. Stored procedures e views

Objeto	Tipo	Uso
sp_criar_paciente	procedure	Chamada em POST /api/pacientes; valida CPF/e-mail duplicados e idade mínima
sp_agendar_consulta	procedure	Chamada em POST /api/consultas; valida paciente, psicólogo e conflito de horário
sp_registrar_evolucao	procedure	Chamada em POST /api/evolucoes; uma evolução por consulta
sp_relatorio_atendimentos	procedure	Relatório por período com REFCURSOR, lendo a view de atendimentos
vw_agenda_completa	view	Consultas + pacientes + psicólogos (JOIN de três tabelas)
vw_evolucoes	view	Prontuário com dados do paciente e do profissional
vw_relatorio_atendimentos	view	Base do relatório, com LEFT JOIN em evoluções
fn_resumo_paciente	function	Totais de consultas e humor médio por paciente
fn_relatorio_atendimentos	function	Versão tabular do relatório, consumida pela API

A procedure de relatório atende ao requisito de utilizar uma **view com JOIN** e não é chamada pela API. Exemplo de uso no PostgreSQL:

```
BEGIN;
CALL sp_relatorio_atendimentos(CURRENT_DATE - 30, CURRENT_DATE, 'relatorio');
FETCH ALL FROM relatorio;
COMMIT;
```

7. Endpoints da API

Todos os dados trafegam em JSON. A tabela abaixo resume as rotas disponíveis (a coleção do Postman acompanha o projeto).

Método	Rota	Descrição
GET	/api/health	Verifica API e conexão com o banco
GET	/api/pacientes	Lista paginada (nome, status, cidade)
GET	/api/pacientes/{id}	Busca paciente por id
GET	/api/pacientes/{id}/resumo	Resumo individual do paciente
POST	/api/pacientes	Cadastra paciente (stored procedure)
PUT	/api/pacientes/{id}	Atualiza paciente
DELETE	/api/pacientes/{id}	Remove paciente
GET / POST	/api/psicologos	Lista / cadastra psicólogos
GET / PUT / DELETE	/api/psicologos/{id}	Consulta / atualiza / remove psicólogo
GET	/api/consultas	Lista consultas com filtros
POST	/api/consultas	Agenda consulta (stored procedure)
GET / PUT / DELETE	/api/consultas/{id}	Consulta / atualiza / remove consulta
GET	/api/evolucoes	Lista evoluções
POST	/api/evolucoes	Registra evolução (stored procedure)
GET / PUT / DELETE	/api/evolucoes/{id}	Consulta / atualiza / remove evolução

Método	Rota	Descrição
GET	/api/relatorios/atendimentos	Relatório por período
GET	/api/relatorios/resumo	Indicadores gerais

Formato padronizado de erro: `{ "erro": true, "mensagem": "...", "status": 422 }`.

8. Testes e evidências

Os testes foram executados no painel web, em uma página de evidências que dispara requisições reais contra a API e na suíte automatizada (pytest). A coleção do Postman também acompanha o repositório.

8.1 Painel web

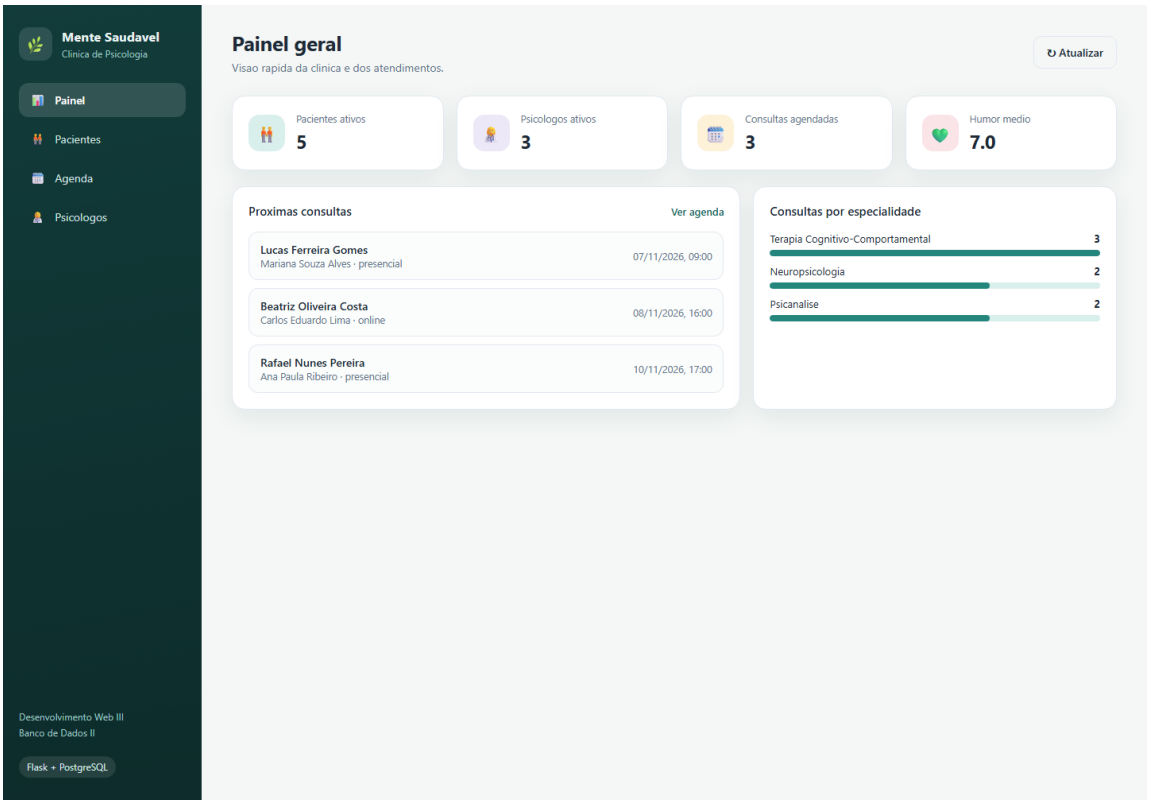


Figura 2 — Painel geral com indicadores, próximas consultas e distribuição por especialidade.

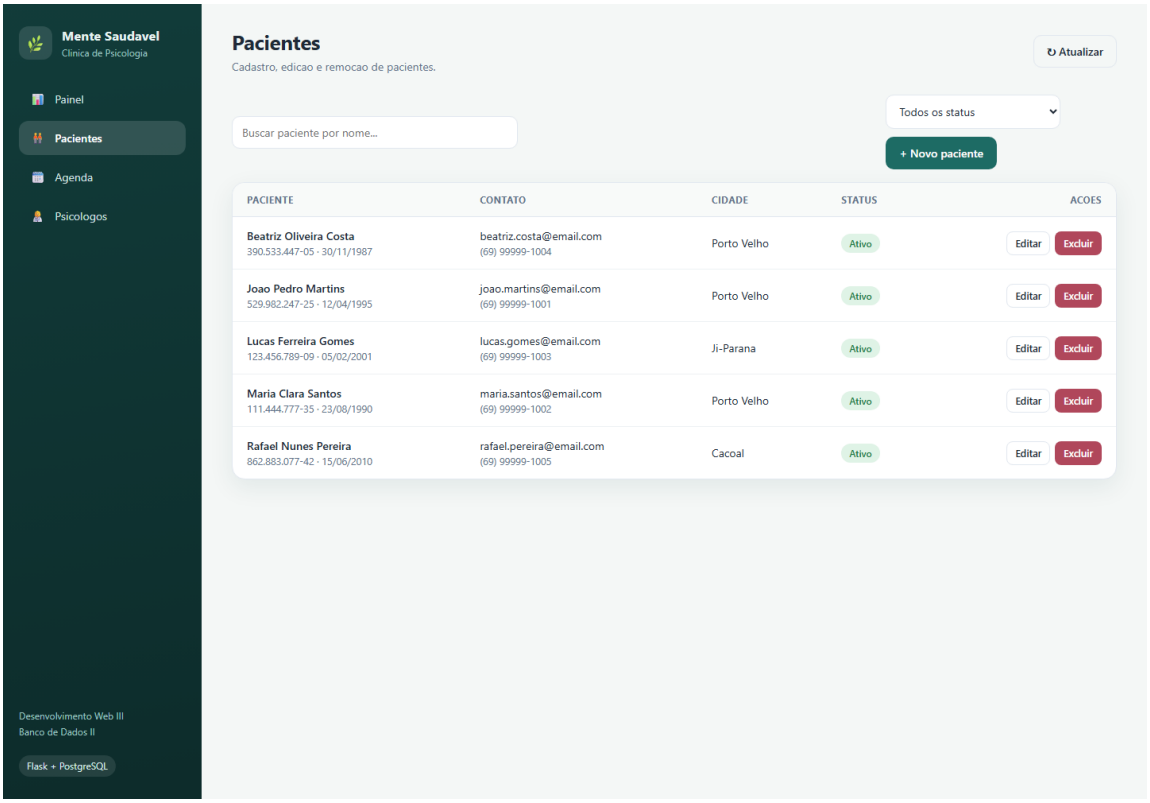


Figura 3 — Tela de pacientes com busca, status e ações de edição/exclusão.

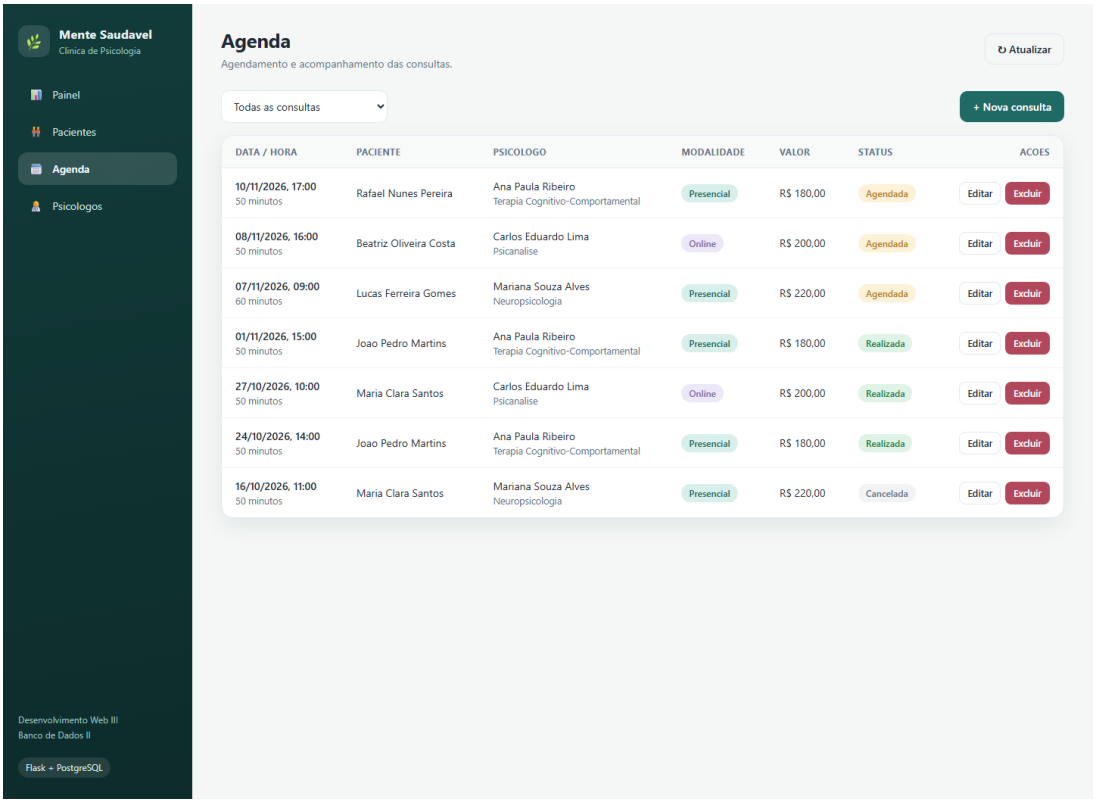


Figura 4 — Agenda de consultas com modalidade, valor e status.

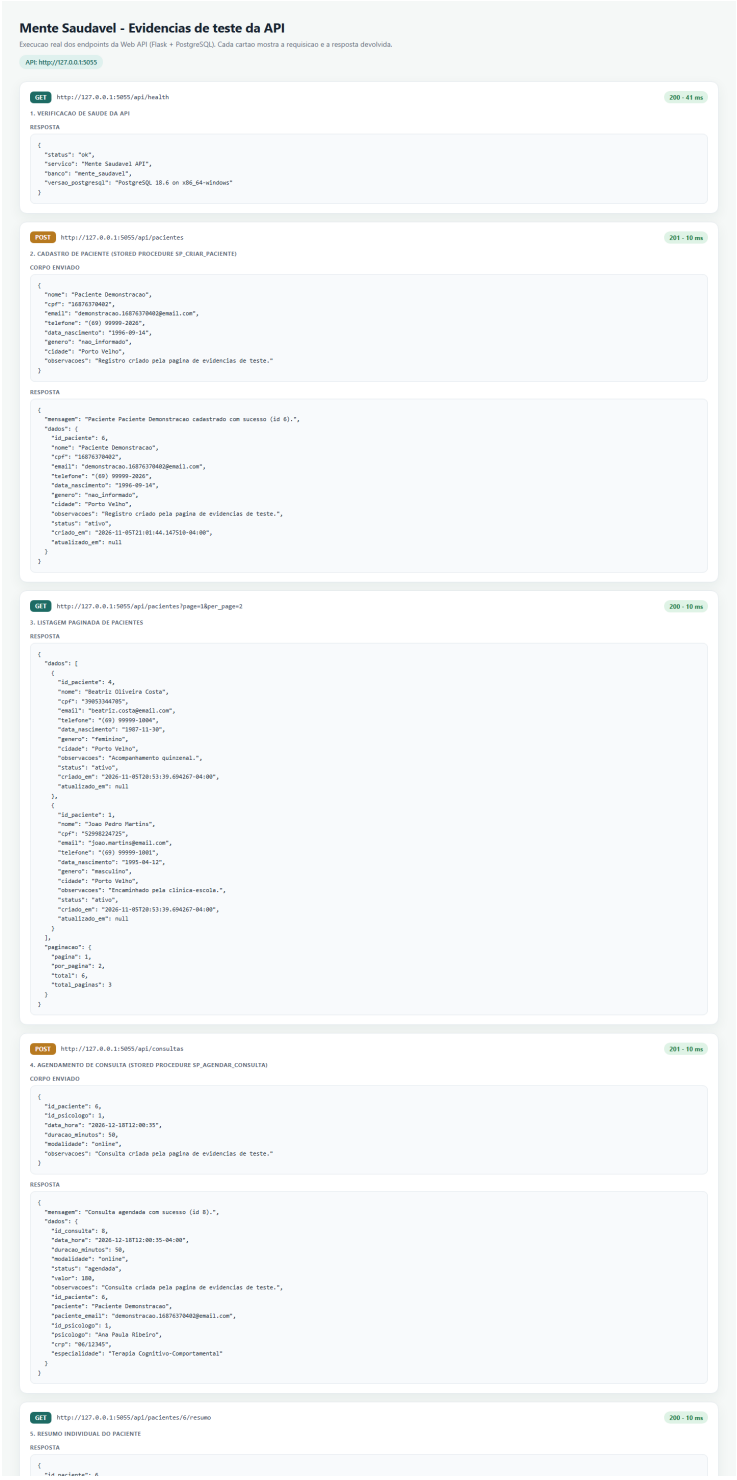


Figura 5 — Execução real dos endpoints: health check, cadastro via stored procedure, listagem, agendamento, resumo e relatório.

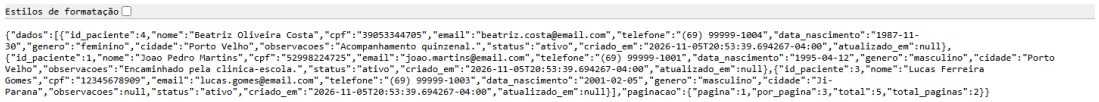


Figura 6 — Resposta JSON da listagem paginada de pacientes.

Suíte automatizada: `pytest -q` — 23 testes aprovados, cobrindo validações de CPF, e-mail, datas, paginação e o fluxo completo de cadastro, consulta, evolução e relatório.

9. Divisão de tarefas

Cada entrega deve ser revisada por pelo menos outro integrante (duplo check), conforme orientação da atividade.

Integrante	Desenvolvimento	Documentação	Revisão
(preencher)	Banco de dados, procedures e views	Script SQL comentado	(outro integrante)
(preencher)	API Flask e validações	Endpoints e testes	(outro integrante)
(preencher)	Frontend e integração	Capturas de tela	(outro integrante)
(preencher)	Testes e relatórios	Documento final	(outro integrante)

10. Como executar

- Criar o banco e executar os scripts na ordem 00, 01, 02, 03 e 04 da pasta `database/`.
- Copiar `.env.example` para `.env` e configurar usuário e senha do PostgreSQL.
- Criar o ambiente virtual e instalar as dependências de `requirements.txt`.
- Executar `python run.py` e acessar `http://localhost:5000`.

As instruções completas, incluindo testes e Postman, estão no `README.md` do repositório.

11. Considerações finais

O projeto aplica, de forma integrada, os conteúdos das duas disciplinas: a API Flask fornece endpoints REST com validações e respostas JSON, enquanto o PostgreSQL garante a integridade

dos dados por meio de chaves, restrições, procedures, views e triggers. A separação em camadas facilita a manutenção e a evolução do sistema.